Swiftsum: Vedic Arithmetic Engine

An Interactive Indian Knowledge Systems (IKS) Computing Engine & 3D Walkthrough

COLLEGE EXHIBITION 2026 | TEAM PRESENTATION GUIDE (8 PRESENTERS) | NEXT.JS 15 + THREE.JS 3D ENGINE | 100% HARDWARE-VERIFIED PROOFS

1. Project Introduction

What is Swiftsum?

Swiftsum is an interactive web-based mathematical platform that revives the computational methods of **ancient Indian mathematics (Vedic Mathematics)**. Instead of treating historical arithmetic as dry theory or rote memory tricks, Swiftsum implements these classical principles as real-time interactive algorithms backed by an animated 3D visual computing environment.

Why We Built Swiftsum (The Problem)

Conventional school arithmetic relies heavily on tedious columnar scratchpad operations: multi-row carries, repeated borrowing across multiple zeroes, and trial-and-error long division. This mechanical approach causes cognitive fatigue and obscures the natural symmetries of the base-10 decimal system.

How Swiftsum Solves It (The Vedic Advantage)

Indian mathematicians recognized that arithmetic is pattern recognition. By utilizing base deviations, 10's complements, and crosswise digit relationships, multi-digit calculations are solved in a single mental line. Swiftsum provides transparent step-by-step proofs and 3D visual coordinate animations for every calculation.

2. Landing Page and User Experience

The user interface was crafted to reflect reverence for historical manuscripts while maintaining state-of-the-art modern responsive performance.

Atmospheric Visual Theme

- **Visual Palette:** Deep obsidian dark theme (`#000000`) illuminated by warm antique-gold highlights (`#E5A93C`) and crisp ivory text (`#F5F0E6`).
- **Cultural Watermarks:** Subtle Devanagari architectural watermarks (**"गणना"** — computation, **"निखिलम्"** — all-from-nine) inspired by ancient stone temple inscriptions.
- **Atmosphere:** Ambient celestial particles and a responsive film-grain texture.

Core Landing Page Sections

- **Hero Section:** Introducing the philosophy of Indian mathematics and rapid access CTAs.
- **Treatise Scrollytelling Journey:** Explores 4 seminal epochs: Sulba Sutras (geometric cords), Pingala (binary prosody), Aryabhata & Brahmagupta (the decimal zero), and Madhava's calculus series.
- **Vedic Math Playground:** A live widget right on the home page where visitors multiply numbers (e.g. 98 × 97) with live base selectors (10, 100, 1000).

Navigation to the Calculators

Visitors navigate seamlessly from the editorial landing page to the dedicated computing workspace (`/calculator`) by clicking the gold navbar button **"Interactive Calculator"** or the prominent CTA **"Launch Full 5-Sūtra Computing Engine"**. The workspace provides five dedicated operations with instant preset loading.

3. The Five Calculators — Mathematical Mechanics (Part 1)

Every calculator in Swiftsum operates on authentic Sanskrit mathematical sūtras. Below are the operational principles, algorithms, and benchmark examples for the first two operations.

CALCULATOR 1: MULTIPLICATION (Nikhilam & Ūrdhva-Tiryagbhyām)

Purpose: Multiplies multi-digit numbers in one mental line without traditional vertical scratchpad tables. Automatically selects *Nikhilam* when numbers cluster near a base of 10, or *Ūrdhva-Tiryagbhyām* (vertical and crosswise) for general arbitrary numbers.

Benchmark Example: Multiply `98 × 97 = 9506` using Nikhilam Base 100

1. **Identify Working Base:** Both 98 and 97 are close to **Base 100** (which has **2 zeros**).
2. **Calculate Deviations:** Deviation of 98 = `98 − 100 = −2`; Deviation of 97 = `97 − 100 = −3`.
3. **Cross-Addition (Left Part):** Add either number to the other's deviation: `98 + (−3) = 95` (or `97 + (−2) = 95`). Both yield exactly **95**.
4. **Multiply Deviations (Right Part):** Multiply the two deviations: `(−2) × (−3) = +6`.
5. **Place-Value Alignment:** Since Base 100 has 2 zeros, the right part must occupy 2 digits. Pad 6 to `"06"`.
6. **Final Synthesis:** Combine Left and Right: `95 | 06` ➔ `9506`. ✓ **VERIFIED: 98 × 97 = 9506**

General Multiplication: For numbers not near a power of 10 (e.g. 43 × 21 = 903), Swiftsum executes Ūrdhva-Tiryagbhyām: units column (3×1=3), crosswise pairs (4×1 + 3×2 = 10 ➔ 0 with carry 1), and tens column (4×2 + 1 = 9) ➔ **903**.

CALCULATOR 2: SQUARING (Yāvadūnam, Ekādhikena & Duplex)

Purpose: Calculates squares instantly using specialized algebraic shortcuts tailored to number patterns.

Method A: Yāvadūnam (Near-Base Squaring)

Sūtra: "Whatever the extent of deficiency, lessen it by that much and set up the square of deficiency."

Benchmark Example: `12² = 144` (Base 10)

1. Base = 10, Deviation `d = +2`.
2. Left side: `Number + d = 12 + 2 = 14`.
3. Right side: `d² = 2² = 4`.
4. Combine: `14 × 10 + 4 = 144`.

✓ **VERIFIED: 12 × 12 = 144**

Also works for negative deviations: `96² = 9216` (Base 100, d = −4 ➔ 96−4 = 92, $(-4)^2 = 16$ ➔ 9216).

Method B: Ekādhikena Pūrveṇa (Ending in 5)

Sūtra: "By one more than the previous one."

Benchmark Example: `85² = 7225` (Ends in 5)

1. Split number into Prefix & Units: `8 | 5`.
2. Find Successor: `8 + 1 = 9`.
3. Left side: `Prefix × Successor = 8 × 9 = 72`.
4. Right side: `5² = 25` (always 25).
5. Combine: `72 | 25` ➔ `7225`.

✓ **VERIFIED: 85 × 85 = 7225**

Method C: General Duplex Squaring (Dvandva Yoga)

For general 2-digit numbers (e.g. `43² = 1849`), Swiftsum implements the Dvandva Yoga method: Tens Duplex `4² × 100 = 1600`, Cross Duplex `2 × (4 × 3) × 10 = 240`, Units Duplex `3² = 9`. Synthesizing: `1600 + 240 + 9 = 1849`.

✓ **VERIFIED: 43 × 43 = 1849**.

3. The Five Calculators — Mathematical Mechanics (Part 2)

CALCULATOR 3: SUBTRACTION (Nikhilam Navataścaramam Daśataḥ)

Purpose: Eliminates error-prone column borrowing across zeroes by converting subtraction into complement addition.

Benchmark Example: Subtract `1000 - 347 = 653` (Power of 10 Base)

1. **Identify Working Base:** The minuend is a power of 10: `Base = 1000`.
2. **Apply Complement Rule:** "All from 9 and the last from 10" on subtrahend 347:
   • Hundreds: `9 - 3 = 6` | • Tens: `9 - 4 = 5` | • Units (last non-zero): `10 - 7 = 3`.
3. **Assemble Complement:** Complement is `653`. Because minuend equals the base, the difference is directly **653**.
4. **Verification by Addition:** `653 + 347 = 1000`. ✓ **VERIFIED**

General Subtraction (e.g. 524 − 189 = 335): Complement of 189 (base 1000) is 811. Add to minuend: 524 + 811 = 1335. Remove reference base 1000: 1335 − 1000 = **335**.

CALCULATOR 4: DIVISION (Nikhilam Vibhāgaḥ & Column Fallback)

Purpose: Performs multi-digit division through complement addition, avoiding trial-and-error multiplications.

Benchmark Example: Divide `1032 ÷ 9 = 114 remainder 6`

1. **Base & Complement:** Divisor = 9, Base = 10, Complement = `10 - 9 = 1`.
2. **Column Partitioning:** Since base has 1 zero, partition dividend into Quotient (103) and Remainder (2): `1 0 3 | 2`.
3. **Propagate Digits Forward:** Bring down 1 ➔ multiply by complement 1 and add to next column: 0 + 1 = 1 ➔ add to next: 3 + 1 = 4 ➔ add to remainder column: 2 + 4 = 6.
4. **Synthesized Output:** Quotient = `114`, Remainder = `6`.
5. **Mathematical Euclidean Proof:** `Dividend = Divisor × Quotient + Remainder`
   ➔ `9 × 114 + 6 = 1026 + 6 = 1032`. Remainder satisfies `0 ≤ 6 < 9`. ✓ **VERIFIED**

CALCULATOR 5: DIVISIBILITY CHECKER (Terminal Digit & Osculators)

Purpose: Evaluates whether a large number divides evenly without performing complete long division.

CRITICAL DISTINCTION: DIVISION VS. DIVISIBILITY CHECK

- **Division** calculates two numbers: a whole *Quotient* and an exact *Remainder* (e.g. 1032 ÷ 9 = 114 rem 6).
- **Divisibility Check** evaluates a single boolean question: *Does the number divide completely with Remainder = 0?*

Case A: 1967 ÷ 5 ➔ NOT DIVISIBLE

1. Rule for 5: A number is divisible by 5 iff terminal units digit ∈ {0, 5}.
2. Terminal digit of 1967 is **7** (7 ∉ {0, 5}).
3. Remainder = `1967 % 5 = 2 ≠ 0`. Verdict: `NOT DIVISIBLE`.

✓ **PROOF: 5 × 393 + 2 = 1967**

Case B: 1000 ÷ 5 ➔ DIVISIBLE

1. Rule for 5: Inspect terminal units digit.
2. Terminal digit of 1000 is **0** (0 ∈ {0, 5}).
3. Remainder = `1000 % 5 = 0`. Verdict: `DIVISIBLE`.

✓ **PROOF: 5 × 200 + 0 = 1000**

4. 3D Walkthrough and Technical Architecture

To make ancient algorithmic thinking captivating, Swiftsum introduces a spatial educational environment where numbers exist as 3D coordinate entities.

Why 3D Animation for Mathematics?

Traditional math software presents flat, static numbers on whiteboards. Swiftsum uses Three.js spatial geometry to turn abstract arithmetic operations into visual choreography:

1. Coordinate Splitting
Numbers physically partition into spatial blocks (e.g. 85 splits into prefix card 8 and terminal card 5).

2. Floating Base Pillars
Reference bases (10, 100, 1000) rise as golden architectural pillars anchoring deviations in 3D space.

3. Particle Vector Rays
Crosswise multiplications shoot luminous particles connecting active digit pairs in real-time.

Interactive 3D Player Features

7-Scene Narrative Structure

Every calculation dynamically compiles into a 7-scene educational sequence:

- **Scene 1:** Introduction & equation setup
- **Scene 2:** Base selection & partition
- **Scene 3:** Deviations / Complements
- **Scene 4:** Cross-addition / Quotient assembly
- **Scene 5:** Product of deviations / Remainder
- **Scene 6:** Result synthesis & combination
- **Scene 7:** Independent verification & proof

Player Controls & Interactivity

- **Scrubber Bar:** Direct scrubbing through scenes 1 to 7.
- **Free Orbit Camera:** Click and drag anywhere in the 3D canvas to rotate the camera around the numbers in 3D.
- **Playback Speed:** Cycle speeds between 0.5×, 1.0×, and 1.5×.
- **Keyboard Shortcuts:** Spacebar (Play/Pause), Left/Right arrow keys (Previous/Next scene), M (Mute narration).
- **Dual View Modes:** Instant toggle between "WATCH 3D EXPLANATION" and "VIEW STEP-BY-STEP SOLUTION".

Technical Architecture (Confirmed Project Stack)

Swiftsum was built from the ground up using modern, high-performance web engineering:

LAYER / COMPONENT	TECHNOLOGY	PURPOSE IN SWIFTSUM
Framework	**Next.js 15.5** (App Router)	Server-rendered static routing, optimized chunk loading, fast page delivery.
Core UI & Logic	**React 19 & TypeScript 5.7**	Strict type safety across math algorithms, responsive state isolation.
3D Graphics Engine	**Three.js** (`^0.186.1`)	Hardware-accelerated WebGL scene graph, 3D typography projection, comet particles.
Styling & Design System	**Tailwind CSS 3.4** & PostCSS	Custom antique-gold color tokens, dark obsidian layout, responsive breakpoints.
UI Motion & Animations	**Framer Motion 12.4** & **GSAP 3.15**	Fluid card morphing, 2D fallback state transitions, scrollytelling scroll progress.
Audio Narration	**Web Speech API** (Browser Native)	Zero-dependency educational voiceover narration synchronized with 3D scenes.
Mathematical Engine	**Pure TypeScript** (Modular)	Zero external math libraries; independent BigInt ground-truth verification routines.

5. Presentation Plan for 8 Team Members

Each presenter has a focused 1-minute speaking role and a live demonstration action. Use natural, conversational English.

Speaker 1: Project Overview — INTRO (1 MIN)

- "Hello everyone. Welcome to **Swiftsum**. Have you ever wondered why school math makes arithmetic feel like a chore?"
- "In ancient India, mathematicians used *sūtras* — short, visual principles based on decimal symmetries to calculate in a single mental line."
- "Swiftsum revives this heritage as an interactive 3D web platform where you don't just get an answer — you see the math happen."

Demo Task: Point to the screen title banner, welcome judges, and introduce the team.

Speaker 2: Landing Page & UX — VISUALS (1 MIN)

- "Our landing page is designed like a digital temple manuscript with dark obsidian tones and antique gold accents."
- "We take visitors through a scrollytelling journey covering the Sulba Sutras, binary patterns, and the discovery of zero."
- "Right on the home page, visitors can test our interactive Vedic Playground before entering the full engine."

Demo Task: Scroll through the ancient book leaf, show the playground, and click 'Launch Full 5-Sūtra Computing Engine'.

Speaker 3: Multiplication — MATH (1 MIN)

- "Let's see the first calculator: **Multiplication**. Watch how fast we multiply 98 × 97."
- "Both numbers are near 100 with deficiencies of −2 and −3. Cross-subtracting gives 95; multiplying the deficiencies gives 06."
- "Instantly, 98 × 97 = 9506, without drawing four rows of scratch work. Swiftsum also verifies this independently."

Demo Task: Click Multiplication tab, select `98 × 97` preset, and toggle to Step-by-Step solution to show vector rays.

Speaker 4: Squaring Specialist — MATH (1 MIN)

- "Next is **Squaring**. Vedic mathematics has specialized shortcuts for different number families."
- "For numbers ending in 5 like 85^2, we use *Ekādhikena Pūrveṇa*: multiply prefix 8 by its successor 9 to get 72, and append 25 ➔ 7225."
- "For 12^2, we use *Yāvadūnam*: 12 + 2 = 14, and $2^2 = 4$ ➔ 144. Our engine also supports general Duplex squaring for numbers like 43^2."

Demo Task: Select Squaring tab, demonstrate `85² = 7225`, then click preset `12² = 144`.

Speaker 5: Subtraction Engine — MATH (1 MIN)

- "Our third calculator is **Subtraction**. The biggest headache in subtraction is borrowing across zeroes like 1000 − 347."
- "Vedic math eliminates borrowing using *Nikhilam*: 'All from 9 and the last from 10'. 9−3=6, 9−4=5, 10−7=3 ➔ answer is 653!"
- "We convert subtraction into simple addition of complements. No crossed-out digits, no mistakes."

Demo Task: Select Subtraction tab, run `1000 − 347 = 653`, and show the reverse addition check `653 + 347 = 1000`.

Speaker 6: Division Specialist — MATH (1 MIN)

- "Our fourth tool is **Division**. Traditional long division requires guessing quotient multiples and subtracting repeatedly."
- "With Nikhilam division, we divide 1032 by 9 using complement addition across columns, assembling Quotient 114 and Remainder 6."
- "Swiftsum verifies every division using Euclid's theorem: Dividend = Divisor × Quotient + Remainder, with Remainder strictly less than divisor."

Demo Task: Select Division tab, run `1032 ÷ 9`, and show the quotient 114, remainder 6, and proof badge.

Speaker 7: Divisibility Checker — MATH (1 MIN)

- "Our fifth tool is the **Divisibility Checker**. It is vital to distinguish division from divisibility."
- "Division calculates the quotient and remainder. Divisibility asks only: *Does the number divide completely with remainder zero?*"
- "For 1967 ÷ 5, the terminal units digit is 7, leaving remainder 2 ➔ Not Divisible. For 1000, last digit is 0 ➔ Divisible."

Demo Task: Select Divisibility tab, show `1967 ÷ 5` (Not Divisible) vs `1000 ÷ 5` (Divisible).

Speaker 8: Tech Stack & Closer — TECH (1 MIN)

- "To bring these ancient sūtras into the modern era, we built Swiftsum using **Next.js 15, React 19, and Three.js** for real-time 3D rendering."
- "Users can orbit around the calculations in 3D, listen to browser voice narration, or inspect verified coordinate proofs."
- "Swiftsum proves that ancient Indian mathematics was deeply logical and algorithmic. We invite the judges to try any numbers!"

Demo Task: Click and drag the 3D player canvas to orbit the camera, and invite judges to enter custom inputs.

6. Likely Judge Questions & Model Answers

Prepare for these questions from exhibition judges and faculty evaluators:

Q1: Why build Swiftsum when ordinary digital calculators already exist?

A: Standard calculators only give the final number — they act as black boxes. Swiftsum is an educational engine that teaches human mental arithmetic techniques through spatial place-value decomposition, helping learners understand *how* and *why* numbers interact.

Q2: Are Vedic mathematics sūtras mathematically rigorous or just shortcuts?

A: They are mathematically rigorous algebraic identities. For example, Nikhilam multiplication is the expansion of $(x + a)(x + b) = x(x + a + b) + ab$ where x is a power of 10. The sūtras organize these algebraic polynomials into rapid mental steps.

Q3: Does Nikhilam multiplication only work for numbers close to 100?

A: Nikhilam works around any power of 10 (10, 100, 1000, etc.). For arbitrary numbers far from any base (like 43 × 21), Swiftsum automatically switches to the general *Ūrdhva-Tiryagbhyām* (vertical and crosswise) sūtra, which handles any multi-digit product.

Q4: How does Swiftsum verify that its calculated answers are 100% correct?

A: Every calculation engine runs an independent ground-truth verification routine using native BigInt arithmetic. Division verifies Euclid's identity (Dividend = Divisor × Q + R, 0 ≤ R < Divisor), subtraction verifies addition (Diff + Subtrahend = Minuend), and squaring verifies standard products.

Q5: What is the fundamental difference between Division and Divisibility Check?

A: Division computes the exact quantitative breakdown: a quotient and a remainder. Divisibility check evaluates whether the remainder is strictly zero, often using fast shortcuts like terminal units inspection or osculators without performing full division.

Q6: Why did you use Three.js for 3D instead of pre-rendered videos?

A: Three.js generates real-time procedural WebGL scenes dynamically for *any custom numbers* the user inputs. Videos would only work for pre-recorded examples. Users can also interactively orbit the camera and scrub time in real time.

Q7: What happens if a user's device or browser does not support WebGL?

A: Swiftsum includes a dedicated `Nikhilam2DFallback` component built with Framer Motion. If WebGL context creation fails, the application automatically falls back to smooth 2D card animations with zero crash.

Q8: What are the current limitations and future scope of the project?

A: Currently Swiftsum focuses on positive integers across five foundational operations. Future extensions will add decimal fractions, square roots (Vargamula), and algebraic polynomial factoring using Vedic sūtras.

7. Quick Revision Table

OPERATION	SANSKRIT SŪTRA	CORE PRINCIPLE	BENCHMARK EXAMPLE	VERIFICATION IDENTITY
Multiplication	Nikhilam / Ūrdhva-Tiryag	Base deviations & cross-addition	`98 × 97 = 9506`	`9506 ÷ 97 = 98`
Squaring	Yāvadūnam / Ekādhikena	Excess addition / Prefix successor	`12² = 144` & `85² = 7225`	`85 × 85 = 7225`
Subtraction	Nikhilam Navataścaramam	All from 9, last from 10 (Complements)	`1000 − 347 = 653`	`653 + 347 = 1000`
Division	Nikhilam Vibhāgaḥ	Deficiency addition across columns	`1032 ÷ 9 = 114 rem 6`	`9 × 114 + 6 = 1032`
Divisibility	Antya Anka / Osculation	Remainder == 0 check (Terminal digit)	`1967 ÷ 5` (No), `1000 ÷ 5` (Yes)	`5 × 393 + 2 = 1967`